



LANGUAGE

Storage Classes & Type Specifiers

auto · extern · register · static
void · const · typedef · volatile

A Complete Visual Guide

What Are Storage Classes?

Storage classes in C define the scope, lifetime, and visibility of a variable or function. They tell the compiler how and where to store a variable.

Scope

Where a variable is accessible
(local, global, file)

Lifetime

How long a variable exists
in memory

Visibility

Who can see and use
the variable

Default Value

The initial value when
not explicitly set

auto — Automatic Storage

The default storage class for all local variables. Entered, destroyed when it exits.

Scope	Local (block)
-------	---------------

Lifetime	Block duration
----------	----------------

Default Value	Garbage (undefined)
---------------	---------------------

Keyword	Optional (implicit)
---------	---------------------



```
#include <stdio.h>

void example() {
    auto int x = 10; /* same as: int x = 10 */
    int y = 20;      /* also auto, implicitly */
    printf("%d %d\n", x, y);
} /* x and y destroyed here */
```

💡 In modern C, writing `auto` is optional — all local variables are `auto` by default.

extern — External Linkage

extern

Declares a variable or function defined in another file — does not allocate memory — just announces existence.

Scope	Global (all files)
Lifetime	Entire program
Default Value	Zero
Use Case	Multi-file programs

```
file.c
int count = 0;

void increment() {
    count++;
}
```



```
file.c
extern int count;
extern void increment();

int main() {
    increment();
    /* count is now 1 */
}
```



extern only declares — the actual definition (memory allocation) must exist exactly once across all files.

register — CPU Register Hint

register

Suggests the compiler store the variable in a CPU register.
Common in loops. Compiler may ignore this hint.

Scope	Local (block)
Lifetime	Block duration
Default Value	Garbage
Address	Cannot take (&)



```
#include <stdio.h>

int sumArray(int arr[], int n) {
    register int i;    /* fast loop counter */
    int sum = 0;
    for (i = 0; i < n; i++)
        sum += arr[i];
    return sum;
}

/* ERROR: &i - cannot take address */
```

💡 Modern compilers optimize automatically. `register` is rarely needed today but is still valid C.

static — Persistent Storage

static

static has TWO distinct uses depending on context:

① Inside a function — Persistent value



```
void counter() {  
    static int count = 0; /* initialized  
once */  
    count++;  
    printf("Called %d times\n", count);  
}  
/* counter() → 1, 2, 3... (value preserved)  
*/
```

② Global / file scope — Limit visibility



```
static int secret = 42; /* only this file */  
static void helper() { /* only this file */  
    /* other files can't see this */  
}  
  
int main() { helper(); }
```

💡 *static local: value survives function calls. static global: limits visibility to current file only.*

void — No Value / Generic Pointer

void has three distinct uses in C:

① Return Type

```
void greet() {  
    printf("Hello!");  
    /* no return value */  
}
```

Function returns nothing

② Parameter List

```
void printHi(void) {  
    /* no parameters */  
    printf("Hi!");  
}
```

Function takes no arguments

③ Void Pointer

```
void *ptr;  
int x = 5;  
ptr = &x; /* generic pointer */
```

Can point to any type



*void * (void pointer) is a powerful feature — it's the basis of generic functions like malloc() and memcpy().*

typedef — Type Aliases

Creates an alias for an existing data type, improving portability.

Without typedef

```
struct Point { int x; int y; };  
struct Point p1;
```

With typedef

```
typedef struct { int x; int y; } Point;  
Point p1; /* cleaner! */
```



```
/* Basic typedef */  
typedef unsigned int uint;  
uint age = 25; /* same as: unsigned int age = 25 */  
  
/* Typedef with struct */  
typedef struct {  
    char name[50];  
    int age;  
} Person;  
  
Person p1; /* no need for 'struct Person p1' */  
p1.age = 30;  
  
/* Typedef for function pointer */  
typedef int (*MathFunc)(int, int);  
MathFunc add = myAddFunction;
```



typedef doesn't create a new type — it just creates a new name for an existing one.

const — Read-Only Values

Makes a variable read-only after initialization.
compile-time error.

const int *p

Pointer can move.

Value it points to cannot change.

int * const p

Pointer is fixed.

Value it points to can change.



```
const int MAX = 100;  
// MAX = 200; ← ERROR: read-only
```

```
/* const pointer examples */  
int x = 10, y = 20;
```

```
const int *p = &x; /* ptr to const int */  
p = &y;           /* OK: pointer moves */  
// *p = 30;      ← ERROR: can't change value
```

```
int * const q = &x; /* const pointer to int */  
*q = 30;           /* OK: can change value */  
// q = &y;        ← ERROR: pointer is fixed
```

💡 Prefer `const` over `#define` — it has type safety and is visible to the debugger.

volatile — Prevent Optimization

Tells the compiler NOT to optimize access to this variable — its value may change unexpectedly (e.g., by hardware or another thread).

✗ Without volatile (bad)



```
int flag = 0;

/* Compiler may optimize loop:
   while(1) { } // stuck forever
   because it thinks flag never
   changes from outside! */
while (flag == 0) {
    /* wait... */
}
```

✓ With volatile (correct)



```
volatile int flag = 0;

/* Compiler re-reads flag from
   memory on every iteration.
   Safe when another thread or
   hardware changes flag */
while (flag == 0) {
    /* wait safely */
}
```



Used in embedded systems, device drivers, and multi-threaded code where hardware can change memory directly.

mutable — Override const (C++ only)

⚠ **mutable is a C++ keyword only. It does not exist in C.**

Allows a member of a const object to still be modified, e.g. for lazy evaluation, or logging counters.

Without mutable

Modifying accessCount inside a const method causes a compile error.

With mutable

accessCount can be modified even inside const methods.

```
class Cache {
public:
    mutable int accessCount = 0; // can change
    int data;

    int getData() const {
        accessCount++; // OK even in const method!
        return data;
    }
};

const Cache c;
c.getData(); /* accessCount increments */
```

💡 *Use mutable sparingly — only for internal state that doesn't affect observable behavior.*

Quick Comparison Table

Keyword	Scope	Lifetime	Default	Key Use
auto	Local	Block	Garbage	Default local var
extern	Global	Program	Zero	Share across files
register	Local	Block	Garbage	Fast loop counters
static	Local/File	Program	Zero	Persist / restrict
void	—	—	—	No value / generic ptr
typedef	—	—	—	Type aliases
const	Any	Depends	—	Read-only values
volatile	Any	Depends	—	Prevent optimization
mutable	Class	Object	—	Override const (C++)

Key Takeaways

auto

All local vars are auto by default — you rarely write it explicitly.

void

Return nothing, accept nothing, or create a generic pointer.

extern

Use to share global variables/functions across multiple .c files.

typedef

Create readable aliases — especially useful with structs and function pointers.

register

A hint only — modern compilers usually optimize better on their own.

const

Protect values from accidental change. Prefer over #define for type safety.

static

Inside function: value persists. At file level: hides from other files.

volatile

Essential in embedded/systems code to prevent dangerous compiler optimizations.